

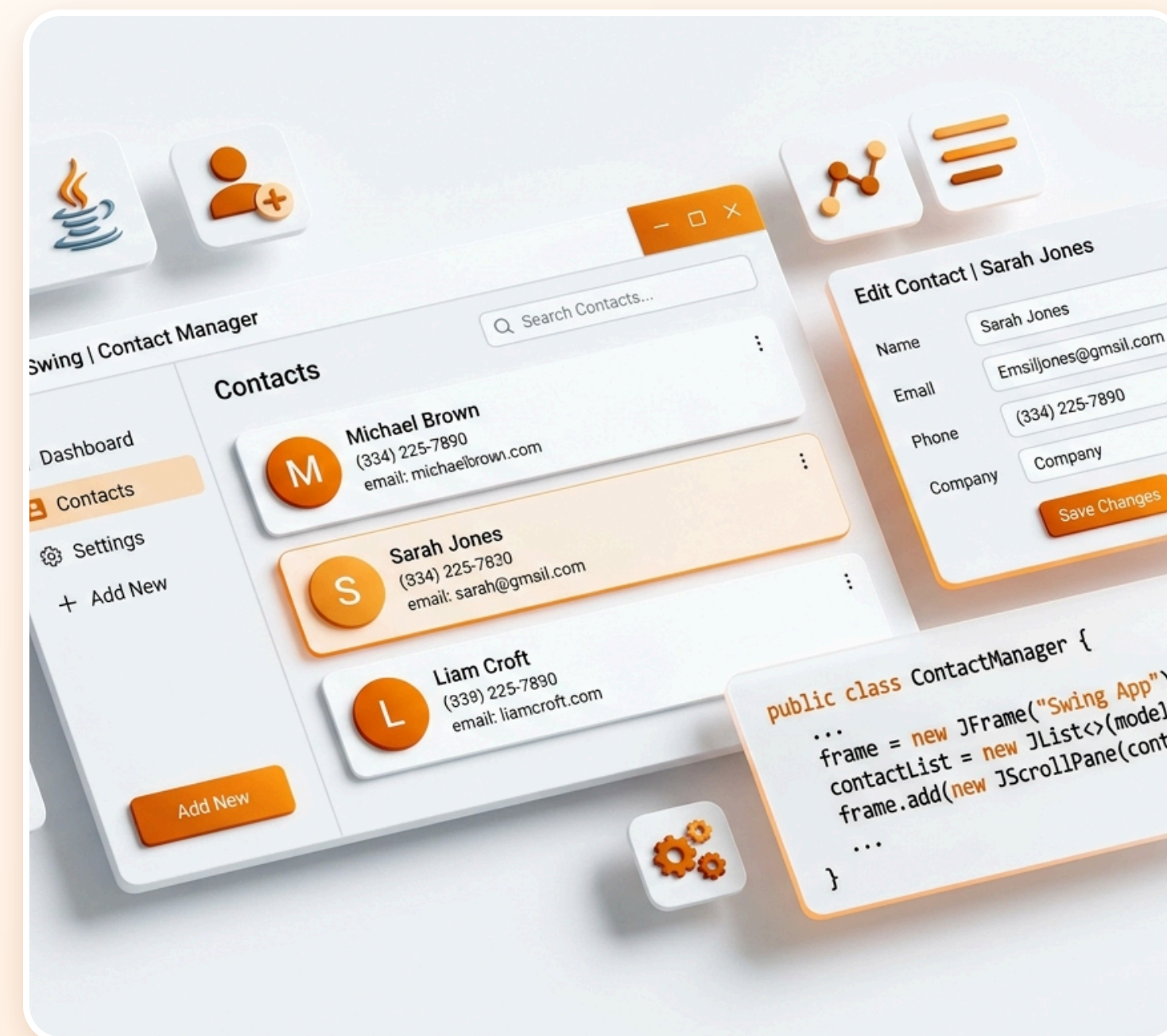
CURSO DE PROGRAMAÇÃO JAVA

INTERFACE GRÁFICA

Construindo Interfaces Desktop com **Java Swing**

Do console à aplicação visual completa: dominando eventos, componentes e tabelas no projeto Agenda de Contatos.

- Java SE & Swing
- NetBeans GUI Builder
- Projeto GerenciaContatos



Do Terminal à Interface Gráfica (GUI)

A grande virada: saindo do fluxo linear para a programação orientada a eventos.

Terminal / Console (Fluxo Linear)

- O programa dita rigidamente o ritmo de execução.
- Execução sequencial em laço `while` (`opcao != 0`).
- O código **pausa e bloqueia** aguardando `Scanner.nextLine()`.
- O usuário fica restrito às opções enumeradas no menu textual.

Interface Gráfica / Swing (Reativo)

- A aplicação desenha a tela e entra em estado de espera.
- O programa permanece vivo graças à **Event Dispatch Thread (EDT)**.
- O usuário dita o ritmo: clica, digita, redimensiona ou fecha.
- Ações disparam **Eventos** que acionam métodos ouvintes (*Listeners*).

Anatomia de uma Aplicação Swing

Hierarquia de componentes: Containers de topo e elementos visuais.

1. Janelas e Containers

JFrame: A janela principal da aplicação com barra de título, botões de fechar e minimizar.

JPanel: Painel intermediário usado para organizar e agrupar componentes.

2. Gerenciadores de Layout

Definem o posicionamento e redimensionamento automático:

FlowLayout, BorderLayout, GridLayout ou GroupLayout (utilizado pelo gerador visual do NetBeans).

3. Componentes Atômicos de Formulário

- **JLabel:** Textos fixos para identificação dos campos.
- **TextField:** Caixas de texto editáveis (Nome, Telefone, Email).
- **Button:** Botões de ação clicáveis (Salvar, Atualizar, Excluir).
- **JTable:** Grade tabular para visualização dos contatos do banco.

O Ponto de Entrada: Criando o JFrame

Como instanciar e exibir a janela com boas práticas e segurança de threads.

```
Tela.java

package gerenciacaocontato.gui;

import javax.swing.JFrame;

public class Tela extends JFrame {

    public Tela() {
        initComponents(); // Constrói componentes visuais
        setLocationRelativeTo(null); // Centraliza no monitor
    }

    public static void main(String[] args) {
        /* Inicializa a GUI dentro da Thread de Eventos (EDT) */
        java.awt.EventQueue.invokeLater(new Runnable() {
            public void run() {
                new Tela().setVisible(true);
            }
        });
    }
}
```

💡 Pontos Essenciais

- `initComponents()`: Inicializa botões, caixas de texto e coordenadas.
- `setLocationRelativeTo(null)`: Evita que a janela abra colada no canto superior da tela.
- `EventQueue.invokeLater`: Garante thread-safety na renderização da interface, prevenindo conflitos gráficos.

Lendo os Campos de Entrada (JTextField)

Resgatando, tratando e sanitizando as strings digitadas pelo usuário.

```
Captura e Validação

// Captura o texto com .getText() e remove espaços extras
String nome = txtNome.getText().trim();
String telefone = txtTelefone.getText().trim();
String email = txtEmail.getText().trim();

// Validação visual preventiva
if (nome.isEmpty()) {
    JOptionPane.showMessageDialog(this,
        "O campo Nome é obrigatório!",
        "Aviso",
        JOptionPane.WARNING_MESSAGE);
    txtNome.requestFocus(); // Devolve o cursor
    return;
}
```

Por que usar `.trim()` ?

Elimina espaços em branco no início e final da digitação, evitando que o aluno salve registros acidentalmente em branco.

Foco com `requestFocus()`

Se a validação falhar, o cursor retorna automaticamente ao campo correspondente, agilizando a correção pelo usuário.

Ouvindo o Clique: ActionListener

Conectando o botão Salvar à regra de negócio do ContatoService.

```
btnSalvarActionPerformed

private void btnSalvarActionPerformed(java.awt.event.ActionEvent evt) {
    String nome = txtNome.getText().trim();
    String telefone = txtTelefone.getText().trim();
    String email = txtEmail.getText().trim();

    // 1. Chama a camada de Serviço (reaproveitada do console)
    contatoService.cadastrarContato(nome, telefone, email);

    // 2. Exibe diálogo de confirmação visual
    JOptionPane.showMessageDialog(this, "Contato salvo com sucesso!");

    // 3. Limpa o formulário para o próximo cadastro
    limparCampos();
}

private void limparCampos() {
    txtNome.setText("");
    txtTelefone.setText("");
    txtEmail.setText("");
    txtNome.requestFocus();
}
```

Separação de Camadas

A tela **não contém SQL** e **não abre conexão JDBC**.

Ela apenas colhe a entrada do usuário e aciona o ContatoService, garantindo reutilização e clareza de código.

Caixas de Diálogo com JOptionPane

Adeus System.out.println: diálogo nativo direto na tela.

```
// 1. Mensagem de Sucesso (Informativa)
JOptionPane.showMessageDialog(
    this,
    "Contato salvo com sucesso!",
    "Sucesso",
    JOptionPane.INFORMATION_MESSAGE
);

// 2. Alerta de Validação
JOptionPane.showMessageDialog(
    this,
    "Por favor, informe um telefone válido.",
    "Aviso",
    JOptionPane.WARNING_MESSAGE
);

// 3. Mensagem de Erro de Banco
JOptionPane.showMessageDialog(
    this,
    "Falha ao conectar ao banco de dados.",
    "Erro de Conexão",
    JOptionPane.ERROR_MESSAGE
);
```

O parâmetro parentComponent

Passar `this` (a própria janela) como primeiro argumento faz a caixa de diálogo abrir **centralizada sobre a janela mãe**, e não no canto do monitor.

Regra de Ouro da Interface Gráfica

O usuário final **não visualiza o console**. Toda comunicação de sucesso, aviso ou falha precisa ser exibida na interface gráfica.

Exibindo Dados: JTable & DefaultTableModel

A separação fundamental entre Componente Visual e Modelo de Dados.

O que é a JTable ?

É a casca visual responsável por desenhar a grade, as linhas e colunas, lidar com o cabeçalho e suportar barra de rolagem (JScrollPane).

O que é o DefaultTableModel ?

É a estrutura em memória que armazena os dados das células. **Nunca adicionamos linhas diretamente na JTable:** nós manipulamos o Modelo!

Fluxo de Atualização da Tabela

1. `contatoService.listarContatos()`
↓ Retorna `List<Contato>`
2. `DefaultTableModel model = (DefaultTableModel) tbl.getModel()`
↓ Limpa com `model.setRowCount(0)`
3. `model.addRow(new Object[]{ id, nome, tel, email })`
↓
✨ **Tela atualizada instantaneamente!**

Preenchendo a Tabela com o Banco

O método `carregarTabela()` passo a passo.

```
carregarTabela()

private void carregarTabela() {
    // 1. Obtém o modelo associado à tabela visual
    DefaultTableModel modelo = (DefaultTableModel) tblContatos.getModel();

    // 2. Limpa linhas anteriores (evita duplicar na tela)
    modelo.setRowCount(0);

    // 3. Busca a lista atualizada do Service
    List<Contato> lista = contatoService.listarContatos();

    // 4. Itera e insere linha a linha no modelo
    for (Contato c : lista) {
        modelo.addRow(new Object[] {
            c.getId(),
            c.getNome(),
            c.getTelefone(),
            c.getEmail()
        });
    }
}
```

Onde chamar este método?

- 1. No construtor da tela:** Logo após o `initComponents()` para abrir a aplicação já com os dados populados.
- 2. Sempre após Salvar, Atualizar ou Excluir:** Mantendo a interface visual 100% sincronizada com o banco.

Selecionando Linhas para Edição

Capturando o clique do mouse na tabela e carregando o formulário.

```
tblContatosMouseClicked

private void tblContatosMouseClicked(java.awt.event.MouseEvent evt) {
    int linha = tblContatos.getSelectedRow();

    if (linha != -1) {
        // Resgata valores da linha selecionada
        String id = tblContatos.getValueAt(linha, 0).toString();
        String nome = tblContatos.getValueAt(linha, 1).toString();
        String telefone = tblContatos.getValueAt(linha, 2).toString();
        String email = tblContatos.getValueAt(linha, 3).toString();

        // Preenche os campos do formulário para edição
        txtId.setText(id);
        txtNome.setText(nome);
        txtTelefone.setText(telefone);
        txtEmail.setText(email);
    }
}
```

Verificação Preventiva

Sempre teste se `getSelectedRow()` `!= -1`. Se o usuário clicar no cabeçalho ou fora de uma linha válida, o retorno será `-1`.

Campo ID Protegido

O campo `txtId` deve ter `txtId.setEnabled(false)` para evitar que o usuário altere a chave primária manualmente!

Exclusão com Diálogo de Confirmação

Evitando acidentes: `JOptionPane.showConfirmDialog`.

```
btnExcluirActionPerformed

private void btnExcluirActionPerformed(java.awt.event.ActionEvent evt) {
    int linha = tblContatos.getSelectedRow();
    if (linha == -1) {
        JOptionPane.showMessageDialog(this, "Selecione um contato na tabela!");
        return;
    }

    // Diálogo de confirmação com opções Sim e Não
    int resposta = JOptionPane.showConfirmDialog(
        this,
        "Deseja realmente excluir este contato?",
        "Confirmar Exclusão",
        JOptionPane.YES_NO_OPTION
    );

    if (resposta == JOptionPane.YES_OPTION) {
        int id = Integer.parseInt(tblContatos.getValueAt(linha, 0).toString());
        contatoService.excluirContato(id);

        carregarTabela(); // Atualiza a grade
        limparCampos();    // Limpa o formulário
    }
}
```

Experiência do Usuário (UX)

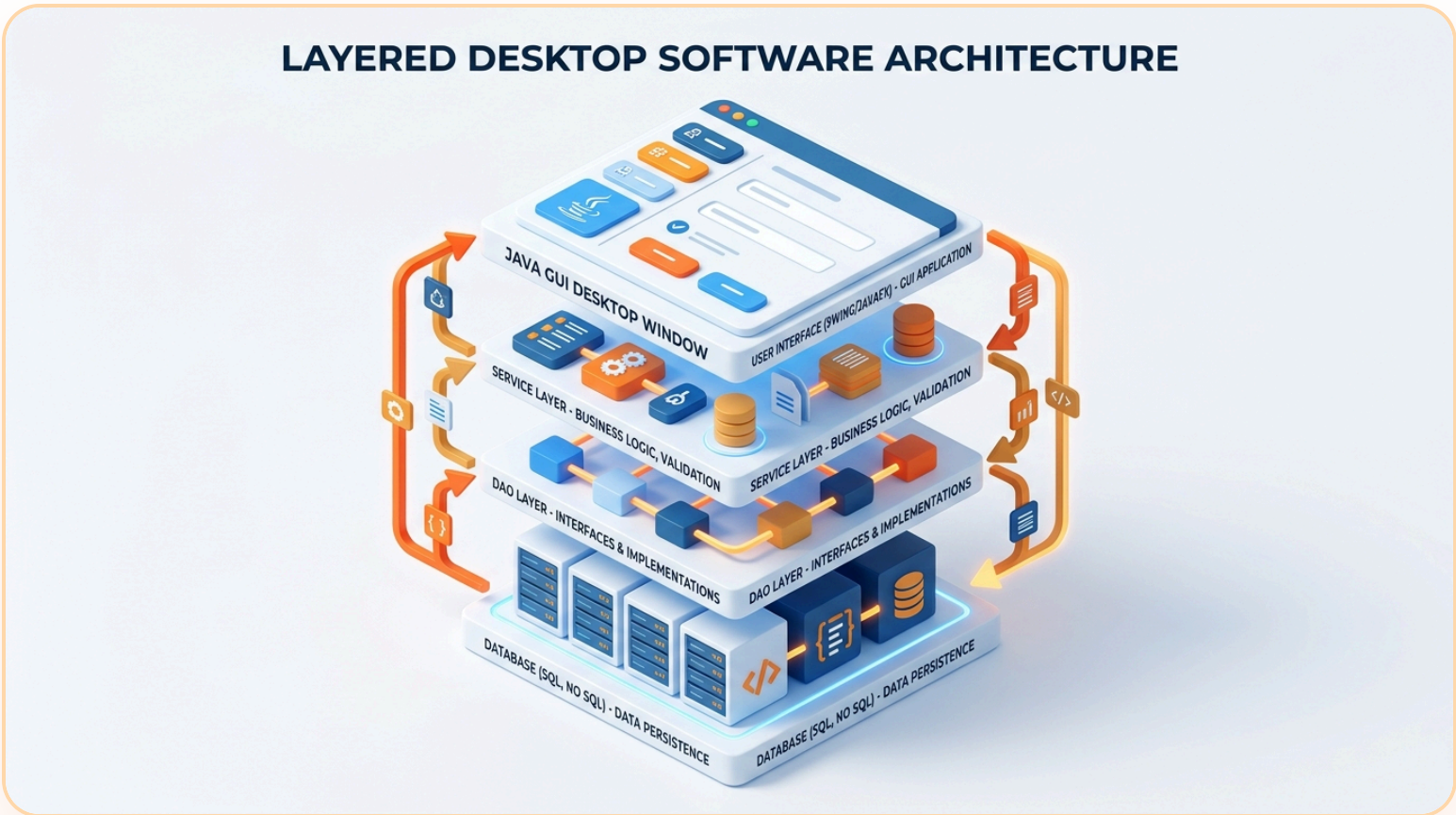
Ações destrutivas nunca devem ser executadas imediatamente no primeiro clique.

O `showConfirmDialog` protege o usuário contra perdas acidentais de dados importantes no banco.

Arquitetura de Software: Separação em Camadas

Por que trocar o console por Swing não exigiu reescrever o banco de dados.

Camada	Responsabilidade no Sistema
View (GUI / Swing)	Captura cliques, digitação e desenha a interface visual.
Service	Validações de regras de negócio e coordenação de fluxo.
DAO	Executa comandos SQL, gerencia conexões e transações.
Model (Contato)	Entidade com atributos, getters, setters e construtores.



Desafio Prático para a Turma

Exercícios para consolidar o aprendizado em sala de aula.

1. Botão "Limpar"

Adicione um botão que limpa todos os campos de texto do formulário, desseleciona a tabela e coloca o foco do cursor em txtNome.

2. Botão "Atualizar"

Permita ao aluno selecionar uma linha, editar os dados nos campos e, ao clicar em "Atualizar", persistir as alterações via `contatoService.atualizarContato(...)`.

3. Desafio Bônus: Controle de Estado dos Botões

Ao abrir a tela, mantenha os botões **Atualizar** e **Excluir** desabilitados (`setEnabled(false)`).

Habilite-os (`setEnabled(true)`) apenas quando o usuário selecionar um contato na tabela!